Real-world (a tool backed by a real API call) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `real_api_tool_practice.py`: a tool that only returns hardcoded data teaches you nothing about the failure modes real tools actually have — timeouts, bad responses, rate limits. This exercise wraps Doc02's `request_with_retry()` inside a `@tool`, so the tool is backed by a *real* network call with real timeout-and-retry handling — which is the whole difference between a simulated tool and one you could ship. **If not:** the Build Task's "one tool that calls a real service" requirement would be the first time you ever put your Doc02 retry logic behind a tool, with no smaller version to trust it against.

Solution

```
# real_api_tool_practice.py
import requests

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


# Doc02's HTTP client with retry, inline so this file runs on its own.
class TransientHTTPError(Exception):
    pass


class PermanentHTTPError(Exception):
    pass


def request_with_retry(method: str, url: str, params: dict = None,
                       retries: int = 3) -> dict:
    for attempt in range(retries):
        try:
            response = requests.request(
                method, url, params=params, timeout=10
            )
        except requests.RequestException as error:
```

```
            if attempt == retries - 1:
                raise TransientHTTPError(str(error))
            continue
        if response.status_code >= 500:
            if attempt == retries - 1:
                raise TransientHTTPError(f"server {response.status_code}")
            continue
        if response.status_code >= 400:
            raise PermanentHTTPError(f"client {response.status_code}")
        return response.json()
    raise TransientHTTPError("request failed after retries")


WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

def build_weather_params(city: str) -> dict | None:
    # why: pulled out on its own so "do we have coordinates for this city"
    # is testable directly, with no real network call or running model.
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return None
    return {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    params = build_weather_params(city)
    if params is None:
        return f"No coordinates on file for {city}."

    try:
        # how: reuses Doc02's request_with_retry() — the tool never calls
        # requests directly, so it gets timeout/retry handling for free.
        response = request_with_retry("GET", WEATHER_URL, params=params)
    except (TransientHTTPError, PermanentHTTPError) as e:
        # why: Doc02's two named errors become a string the model can
        # read — the tool never crashes the program
        return f"Error: the weather service failed: {e}"
```

```
    temperature = response["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"

# Ask the model; it decides whether to call get_weather.
model = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([get_weather])
response = model.invoke("What's the weather in Lahore?")

if not response.tool_calls:
    print("The model answered directly, without calling a tool.")
    print(response.content)
else:
    for call in response.tool_calls:
        if call["name"] == "get_weather":
            result = get_weather.invoke(call["args"])
            print(f"Called get_weather({call['args']}) -> {result}")
        else:
            print(f"Unexpected tool requested: {call['name']}")
```

Expected output:

```
Called get_weather({'city': 'Lahore'}) -> Currently 34.2°C in Lahore
```